APCS 程式設計系列 · U09 · 實作中高級 · C++版

堆疊、佇列與內建資料結構 詳解

stack 後進先出 · 括號配對與後序運算式 · queue 先進先出 · deque · 優先佇列 · map / set 計數去重 · 陣列模擬

Part A 答案速查

1	2	3	4	5	6	7	8	9	10
B	A	C	D	A	B	A	C	D	C
11	12								
B	D								

Part A 程式識讀詳解

第 1 題 答案：(B)

- i = 2 時放進 2 又馬上拿出，印 2；i = 4 時放進 4 又拿出，印 4。
- 最後堆疊裡剩 1、3（3 在上面），後進先出，依序印 3、1。

【錯】選 2 4 1 3 是把剩下的部分當成佇列（先進先出）。

第 2 題 答案：(A)

- 丟 1、把 2 移到最後；丟 3、把 4 移到最後；丟 5、把 6 移到最後，剩 2 4 6。
- 丟 2、把 4 移到最後，剩 6 4；丟 6、把 4 移到最後，最後剩 4。

第 3 題 答案：(C)

- 後序：先寫兩個運算元，再寫運算符號。3 + 4 → 3 4 +；5 - 2 → 5 2 -；兩個結果相乘，* 放最後。
- 用堆疊驗算：3 4 + 得 7，5 2 - 得 3，最後 7 * 3 = 21，與原式相同。

【錯】* + 3 4 - 5 2 是「前序」（符號寫在前面）。

第 4 題 答案：(D)

- 第一次：b = 1、a = 4，放回 4 - 1 = 3，堆疊變成 9 3。
- 第二次：b = 3、a = 9，印出 9 - 3 = 6。這就是後序運算式 9 4 1 - - 的算法。

【錯】把先拿出來的當左邊（算 b - a），就會得到負數。

第 5 題 答案：(A)

- 要先取出 3，必須先放 1、2、3；這時 2 壓在 1 上面。
- 下一個能取出的只有 2（或再放 4 進來），不可能直接拿到 1。

【錯】2 1 4 3：放 1、2，取 2、取 1，放 3、4，取 4、取 3，是可能的。

第 6 題 答案：(B)

- priority_queue 預設是「大的先出」，與放入順序無關。
- 先取到最大的 8，pop 之後最大的是 5。

【錯】選 1 3 是 Python heapq（小的先出）的規則。

第 7 題 答案：(A)

- top 從 -1 開始，++top 是先加再存：7、2、9 存在 st[0]、st[1]、st[2]，top = 2。
- top-- 之後 top = 1，st[1] = 2。

【錯】以為 top 是元素個數（3），減 1 之後是 2，就會選 2 9。

第 8 題 答案：(C)

- 佇列滿了（n == 4）就先把最前面丟掉，再放新的：永遠保留「最後 4 個」，也就是 4、5、6、7。
- 最前面是 4，最後面是 7。陣列實際內容是 q = {5, 6, 7, 4}，f = 3。

【錯】直接看陣列 q[0] 是 5，就會選 5 7；環狀佇列的開頭在 f，不在索引 0。

第 9 題 答案：(D)

- 出現次數：1 出現 2 次、2 出現 2 次、3 出現 3 次、5 出現 1 次。
- 次數最多的是 3（3 次），印出「數字 次數」：3 3。

第 10 題 答案：(C)

- 每個數只在「第一次出現」時印出，之後再出現就跳過。
- 所以保留原本出現的順序：4、7、1、9（不是由小到大）。

第 11 題 答案：(B)

- 只有一種括號時，d 就是「堆疊裡有幾個左括號」。
- d 依序是 1 2 1 2 3 2 1 0 1 0：最深 3 層，最後 d = 0（全部配對成功）。

第 12 題 答案：(D)

- 最近看過的最先被拿出來，就是「後進先出」。
- 佇列是先進先出，會回到「最早」看的頁面。

Part B 程式實作參考解答

實作 1 括號檢查器

解題思路

- 左括號放進堆疊，堆疊大小的最大值就是最深層數。
- 右括號：堆疊是空的、或頂端不是同一種左括號 → 不合法；否則拿掉頂端。
- 全部讀完後，堆疊必須是空的才合法。

【錯】只數左右括號個數相不相等：([)] 個數相等卻不合法，一定要用堆疊比對種類。

【錯】忘了最後檢查堆疊是否為空：(這種只有左括號的會被誤判成合法。

C++ 括號檢查器

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    string s;
    cin >> s;
    stack<char> st;
    int depth = 0;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            st.push(c);
            depth = max(depth, (int)st.size());
        } else {
            char want = c == ')' ? '(' : c == ']' ? '[' : '{';
            if (st.empty() || st.top() != want) {
                cout << "No\n";
                return 0;
            }
            st.pop();
        }
    }
    if (st.empty()) cout << "Yes " << depth << "\n";
    else cout << "No\n";
}
```

時間複雜度：O(|S|)

實作 2 後序計算機

解題思路

- 讀到數字：放進堆疊。
- 讀到運算符號：先拿出 b（右邊）、再拿出 a（左邊），算 a op b 放回去。
- 全部處理完，堆疊裡剩下的唯一一個數就是答案。

【錯】先拿出來的是右邊：2 10 - 要算 2 - 10 = -8，寫反會得到 8。

【錯】10^4 連乘四次就是 10^{16}，超過 int，堆疊要存 long long。

C++ 後序計算機

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    stack<long long> st;
    for (int i = 0; i < n; i++) {
        string t;
        cin >> t;
        if (t == "+" || t == "-" || t == "*") {
            long long b = st.top(); st.pop();   // 右邊
            long long a = st.top(); st.pop();   // 左邊
            if (t == "+") st.push(a + b);
            else if (t == "-") st.push(a - b);
            else st.push(a * b);
        } else {
            st.push(stoll(t));
        }
    }
    cout << st.top() << "\n";
}
```

時間複雜度：O(n)

實作 3 櫃台叫號系統

解題思路

- 前後都要放、都要拿，正好是 deque（雙端佇列）：IN 放後面、VIP 放前面、OUT 拿前面、LEAVE 拿後面。
- OUT 和 LEAVE 之前都要先檢查隊伍是不是空的。
- 輸出很多行，用 "\n" 換行並關閉同步，速度才夠。

【錯】用 vector 的 insert(begin()) / erase(begin()) 模擬：每次 O(n)，2×10^5 個指令會超時。

【錯】隊伍空的時候 LEAVE 什麼都不做，不要輸出任何東西。

C++ 櫃台叫號系統

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int m;
    cin >> m;
    deque<int> dq;
    string cmd;
    for (int i = 0; i < m; i++) {
        cin >> cmd;
        if (cmd == "IN") { int x; cin >> x; dq.push_back(x); }
        else if (cmd == "VIP") { int x; cin >> x; dq.push_front(x); }
        else if (cmd == "OUT") {
            if (dq.empty()) cout << "EMPTY\n";
            else { cout << dq.front() << "\n"; dq.pop_front(); }
        } else if (!dq.empty()) dq.pop_back();   // LEAVE
    }
    cout << dq.size() << "\n";
}
```

時間複雜度：O(m)，每個指令 O(1)

實作 4 急診叫號

解題思路

- 每次都要「目前最大」→ 優先佇列，放入、拿出都是 O(log n)。
- 同嚴重度先到先看：把「到達順序」也放進去一起比較。
- C++ 的 priority_queue 大的先出：放 (嚴重度, -到達順序)，越早到的負號後越大，越先出來。

【錯】放 (嚴重度, 到達順序)：同分時變成「晚到的先出」，因為 priority_queue 大的先出。

【錯】每次 CALL 都把等候名單整個排序：O(m^2 log m)，一定超時。

C++ 急診叫號

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int m, v;
    cin >> m;
    priority_queue<pair<int, int>> pq;   // (嚴重度, -到達順序)
    vector<string> name;
    string cmd, s;
    for (int i = 0; i < m; i++) {
        cin >> cmd;
        if (cmd == "ADD") {
            cin >> s >> v;
            pq.push({v, -(int)name.size()});
            name.push_back(s);
        } else if (pq.empty()) cout << "NONE\n";
        else {
            cout << name[-pq.top().second] << "\n";
            pq.pop();
        }
    }
}
```

時間複雜度：O(m log m)

實作 5 班代表選舉

解題思路

- 用 map<string, int> 計數：cnt[name]++。
- 把 (-票數, 名字) 放進 vector 排序：票數取負號，小的在前就變成票數多的在前；同票再比名字。

【錯】用 unordered_map 計數後直接走訪輸出：順序不固定，一定要排序。

【錯】同票時要比名字：ab 排在 abc 前面，abc 排在 b 前面（字典序逐字比較）。

C++ 班代表選舉

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    map<string, int> cnt;
    for (int i = 0; i < n; i++) {
        string s;
        cin >> s;
        cnt[s]++;
    }
    vector<pair<int, string>> v;
    for (auto& [s, c] : cnt) v.push_back({-c, s});   // 票數取負號
    sort(v.begin(), v.end());
    cout << v.size() << "\n";
    for (auto& [c, s] : v) cout << s << " " << -c << "\n";
}
```

時間複雜度：O(n log n)

自我診斷表

對完答案，數一數每個觀念錯幾題：全對圈綠、錯 1 題圈黃、錯 2 題以上圈紅。紅燈的觀念回去看講義那一段。

觀念	題號	錯幾題	燈號（圈一個）	回去看哪裡
堆疊操作	A1、A5		綠 黃 紅	講義第 1 段
括號配對	A11、B1		綠 黃 紅	講義第 1 段「括號配對」
後序運算式	A3、A4、B2		綠 黃 紅	講義第 2 段（先拿出來的是右邊）
佇列模擬	A2、B3		綠 黃 紅	講義第 3 段
優先佇列	A6、B4		綠 黃 紅	講義第 4 段
計數與去重	A9、A10、B5		綠 黃 紅	講義第 5 段
陣列與 list 模擬	A7、A8		綠 黃 紅	講義第 6 段
資料結構選擇	A12		綠 黃 紅	講義第 1～5 段：後進先出、先進先出、最優先